

# Effectful Computations with Future Conditions: A Monadic Approach to Temporal Specification

AUTHOR NAME, Institution, Country

Modular specification is local: a pre/post pair, a type, a contract each describe one boundary at one moment. Temporal obligations are not local — when a mutex is acquired, the release that balances it lies in a context the specifier cannot see. Existing answers give up modularity (a global runtime monitor) or bake the obligation into the type, so a change to the protocol perturbs type signatures throughout the program (linearity, typestate).

We observe that sequencing a computation against an obligation about its future is *subtraction*: a continuation that partially discharges an obligation leaves a *residual* naming exactly what is still owed, and residuals compose. We present *Pledge*, a Haskell library built on one composition law. The law needs six operations — concatenation, meet, a unit, a top, and two quotients — to propagate obligations through sequential composition; we call this six-operation class *Composable*.

We instantiate this law for regular expressions, composed via Brzozowski derivatives, and build three ways of putting *Pledge* to work on top of it: an LTL layer that reads its residual as a compositional runtime monitor; a second, SMT-checked instance, GRE, that pairs it with a Presburger-arithmetic predicate discharged by Z3 and checked over symbolic states, giving *Pledge* a static-verification mode; and a bounded MTL layer that adds discrete-time intervals, compiled down by finite unrolling.

We prototype this composition law as a monad in Haskell, so obligations become ordinary values threaded by  $\gg$  over an arbitrary base monad, with residuals computed as its real effects run. We prove the monad laws in Lean 4 from fourteen algebraic axioms over *Composable*. The RE instance of *Composable* satisfies them under exactly two side conditions: ??? . A soundness theorem ties the two readings together: a discharged residual entails that the trace a run emitted met every obligation its primitives named.

CCS Concepts: • **Software and its engineering** → **Functional languages**; *Data types and structures*; *Software verification and validation*.

Additional Key Words and Phrases: temporal specification, monadic effects, regular expressions, Brzozowski derivatives, left-quotient, right-quotient, Presburger arithmetic, metric temporal logic, runtime monitoring, static verification, Haskell, Lean 4

## ACM Reference Format:

Author Name. 2026. Effectful Computations with Future Conditions: A Monadic Approach to Temporal Specification. *Proc. ACM Program. Lang.* 1, HASKELL (October 2026), 21 pages.

## 1 Introduction

Pre- and post-conditions are the workhorse of modular program specification. A pre-condition constrains the state in which an operation may be invoked; a post-condition describes what holds immediately after it returns. Yet many correctness properties span a *sequence* of operations, with arbitrary computation in between.

Consider three scenarios from the Haskell ecosystem: (i) *Mutex discipline*. An `acquire(m)` must eventually be followed by `release(m)` on every code path. A post-condition on `acquire` records the

---

Author's Contact Information: Author Name, Institution, City, Country, [author@institution.edu](mailto:author@institution.edu).

---

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2475-1421/2026/10-ART

<https://doi.org/>

acquired state, but the release that discharges it lies outside the scope that post-condition describes. (ii) *Database savepoints*. A `beginTx` must eventually be committed or rolled back; a pre/post pair on the call does not carry that obligation forward through the intermediate code, so it must be re-stated by hand at every step in between. (iii) *File handles and heap cells*. Opening a file in read-mode must be followed by a close; a heap cell returned by `malloc` must eventually be freed, and not freed twice.

*Existing remedies and their limits*. The bracket function lexically scopes cleanup but cannot propagate obligations beyond the lexical scope. `ResourceT` [?] threads cleanup through every bind but models only the cleanup half of an obligation, discharging it uniformly at scope exit. Linear Haskell [?] enforces “consume exactly once”, which does force discharge along every branch, but the obligation is structural: it fixes *that* a resource is consumed, not the order of the operations leading there, and carries no quantitative reading. Parameterised and indexed monads [?] encode protocol states at the type level but require non-standard syntax and are confined to a single domain.

*The gap*. Future conditions themselves are not new. ? introduce them as a first-class extension to pre/post contracts and give a logical semantics; ? encode temporal properties as regular expressions and use derivatives to drive automated program repair in the ProveNFix tool. Neither, however, treats an obligation as a *value* that a program can carry, combine, and hand on. The first works at the level of a logical system, with no implementation; the second embeds its reasoning inside a whole-program analysis that a programmer cannot invoke compositionally, and that is specific to traces. What is missing is an account of how an obligation propagates through effectful code one bind at a time — and, with it, a library a Haskell programmer can actually use.

*Why a library, and not a type system or a model checker*. Temporal properties have been attacked from many directions: encoded in indexed or graded types, discharged by a model checker, or proved in a theorem prover. Each buys stronger guarantees at a cost. Type-level encodings force the protocol into the type of every intermediate computation, so a change to the specification perturbs type signatures throughout the program. Model checkers and provers work on a model of the program rather than the program, and demand that the property be fixed before the code is written. Making the obligation an ordinary Haskell *value* costs the static guarantee, but buys three things those approaches do not offer together: obligations can be computed — built from runtime data, passed to functions, stored in data structures; one bind formula serves every domain, so a new obligation language is a new **Composable** instance rather than a new encoding; and the unmet remainder is itself an inspectable value, so a failure *names* what is still owed instead of reporting that a proof did not go through.

*What Pledge does, and what it does not*. A **Pledge**  $m \text{ eff}$  a value is a computation in the underlying monad  $m$  that additionally produces three annotation fields (*pre*, *post*, *fut*)  $:: \text{eff}$ , and composing such values propagates those annotations algebraically alongside whatever  $m$  does. Two consequences are worth stating before the details.

First, the annotations are *path-sensitive*. Because  $m$  is a monad, bind must run the first action before it can apply the continuation to the result, so the annotations that come out describe the one path the program actually took — not an over-approximation of every path it might have taken. This is what lets §5 read **Pledge** as a *runtime monitor*: the residual after each bind is the monitor’s state, computed as real effects run rather than against a static model.

Second, nothing intervenes, as the library stands today. The residual is a value to be inspected; a program that fails to discharge an obligation still runs to completion, and **Pledge** names the omission rather than preventing it. Promoting the residual to a runtime guard is future work we scope precisely in §8.

*Our approach.* A value of type `Pledge m eff a` carries three annotations: a precondition  $pre :: eff$  (what must have held before), a postcondition  $post :: eff$  (what this action produces), and a future condition  $fut :: eff$  (what must hold afterwards). Monadic bind propagates these through two quotient operations on `eff`:

$$pre(p \gg q) = pre(p) \sqcap (pre(q) / post(p)) \quad (1)$$

$$post(p \gg q) = post(p) \cdot post(q) \quad (2)$$

$$fut(p \gg q) = (fut(p) \setminus post(q)) \sqcap fut(q) \quad (3)$$

A fully-discharged residual —  $pre = \neg\emptyset$  and  $fut = \neg\emptyset$  — is the certificate that the composed program is contract-correct; anything else names what is still owed. §2 develops this formula and the six-operation `Composable` algebra it is stated over; §3 identifies exactly what that algebra must satisfy for Equations (1)–(3) to obey the monad laws; §4 instantiates the algebra with regular expressions and Brzozowski derivatives and proves the instance sound. On top of that instance, §5 reads its residual as a compositional runtime monitor via an LTL layer, §6 builds a second, SMT-checked instance GRE for static verification, and §7 adds a bounded MTL layer with discrete-time intervals; §8–§10 lay out, concretely, the three pieces of engineering the design still needs.

*Contributions.*

- (1) **The `Pledge` monad transformer and the `Composable` algebra** (§2): six operations that make future conditions first-class Haskell values propagated by a single bind formula, closed under products and pointwise lifting through any monad.
- (2) **Mechanised metatheory, and its exact boundary** (§3–§4). We identify fourteen algebraic axioms sufficient for the three monad laws on the  $(ret, post, fut)$  triple and prove the laws sorry-free in Lean 4. Mechanising the RE instance against those axioms then *delimits* them, and we state and prove the two side conditions — single-word postconditions, past-closed preconditions — that restore full equality, together with a soundness theorem: a fully discharged RE residual entails that the emitted trace met every obligation its primitives named.
- (3) **A runtime monitor on top of RE** (§5): an LTL layer that reframes `Pledge` as a compositional runtime monitor whose state is the residual itself, compiled to RE so §4’s theory applies unchanged.
- (4) **A second, SMT-checked instance** (§6): GRE pairs an RE with a Presburger predicate over heap addresses, discharged by Z3, and because that predicate is checked symbolically over addresses rather than a concrete run, it gives `Pledge` a static-verification mode beside LTL’s runtime one.
- (5) **A bounded temporal layer on top of RE** (§7): an MTL layer that adds discrete-time intervals, compiled down to RE by finite unrolling so §4’s theory applies unchanged.
- (6) **A concrete roadmap for the unfinished third of the design** (§8–§10): making a violation’s location traceable, automating the Lean proof obligations a new instance incurs, and the empirical evaluation the design has not yet had.

## 2 What Is Pledge

### 2.1 The Composable Class

We abstract the specification algebra via a type class with six operations:

```
class Composable a where
  concatenation :: a → a → a    -- (·), identity: empty
  conjunction  :: a → a → a    -- (⊓), identity: universe
  empty        :: a             -- identity for (·)
```

```

universe      :: a          -- identity for ( $\sqcap$ )
leftQuotient  :: a  $\rightarrow$  a  $\rightarrow$  a -- dividend  $\setminus$  divisor
rightQuotient :: a  $\rightarrow$  a  $\rightarrow$  a -- dividend  $/$  divisor

```

The library provides Unicode infix aliases:  $(\cdot)$  = **concatenation**,  $(\sqcap)$  = **conjunction**,  $a \setminus b$  = **leftQuotient**  $b$   $a$ , and  $p / q$  = **rightQuotient**  $q$   $p$ . The swapped arguments in the infix definitions are intentional:  $fut \setminus post$  means “left-quotient of  $fut$  by  $post$ ,” i.e., strip  $post$  from the left of  $fut$ ;  $pre / post$  means “right-quotient of  $pre$  by  $post$ ,” i.e., strip  $post$  from the right of  $pre$ .

*Algebraic roles.* **concatenation** sequences effects; **conjunction** combines simultaneous constraints; **empty** and **universe** appear in pure. The two quotient operations implement *obligation discharge*:  $fut \setminus post$  asks what of the original future obligation remains unmet once the continuation has posted  $post$ , and  $pre / post$  asks what portion of the next action’s precondition is not yet satisfied by the current action’s postcondition. Two base cases anchor both quotients:  $r \setminus \varepsilon = r$  (nothing observed, obligation unchanged) and  $r \setminus \top = \top$  (universal postcondition covers anything). The symmetric laws hold for  $/$ .

*Composing instances.* The instance for pairs lifts **Composable** component-wise, and a separate instance lifts it pointwise through any Applicative functor  $m$ , so **Composable**  $(m \text{ eff})$  holds whenever **Composable**  $\text{eff}$  does:

```

instance (Composable a, Composable b)  $\Rightarrow$  Composable (a, b) where
  concatenation (a1,b1) (a2,b2) = (concatenation a1 a2, concatenation b1 b2)
  leftQuotient  (a1,b1) (a2,b2) = (leftQuotient  a1 a2, leftQuotient  b1 b2)
  -- etc.

```

Independent specification dimensions — a trace obligation and a heap obligation, say — combine by pairing their **Composable** instances; §6 instead gives a fourth instance that couples the two dimensions directly.

## 2.2 The Pledge Monad Transformer

```

newtype Pledge m eff a = Pledge { runPledge :: m (a, eff, eff, eff) }

```

A value of type **Pledge**  $m \text{ eff}$   $a$  is a computation in the underlying monad  $m$  that produces a 4-tuple  $(a, pre, post, fut)$ : the return value and three annotation fields. When  $m = \text{IO}$ , the annotations can be built from actual runtime values (e.g., a freshly allocated file handle), because the 4-tuple is produced by the  $\text{IO}$  action itself.

Two combinators support interoperability:

```

-- lift a plain m-action with trivial annotations
liftPledge :: (Composable eff, Applicative m)  $\Rightarrow$  m a  $\rightarrow$  Pledge m eff a
liftPledge ma = Pledge $ fmap (, universe, empty, universe) ma

-- run once and collect all four components
inspect :: Functor m  $\Rightarrow$  Pledge m eff a  $\rightarrow$  m (PledgeResult eff a)
inspect (Pledge ma) = fmap (\(a,pre,post,fut)  $\rightarrow$  PledgeResult a pre post fut) ma

```

**liftPledge** embeds any  $m$ -action as a **Pledge** with no precondition, no postcondition, and no future obligation. **inspect** runs the action exactly once and packages the result; callers should prefer **inspect** over the individual field accessors (`getPre`, `getPost`, `getFut`) when  $m$  has side effects.

### 2.3 Monad Instance

`pure` introduces a value with no effect and no obligation, and bind sequences two computations and propagates annotations according to Equations (1)–(3):

```
pure x = Pledge $ pure (x, universe, empty, universe)

Pledge ma >>= g = Pledge $ do
  (a, preA, postA, futA) <- ma
  (b, preB, postB, futB) <- runPledge (g a)
  return (b, preA  $\sqcap$  (preB / postA),
          postA  $\cdot$  postB,
          (futA  $\setminus$  postB)  $\sqcap$  futB)
```

When the future component normalises to  $\top$ , all future obligations are discharged. When  $post(A)$  does not cover  $pre(B)$ , the residual precondition is  $\emptyset$ , flagging the violation; when it fully covers  $pre(B)$ , the residual is  $\varepsilon$  and the combined precondition reduces to  $pre(A)$ , matching the standard Hoare sequencing rule. §3 shows that, despite this asymmetric phrasing, all four fields are treated by exactly the same algebra, and identifies the condition under which the triple  $(ret, post, fut)$  satisfies the monad laws with equality.

*One operation, two directions.* The two quotients are not independent machinery. A future condition is an obligation on the trace still to come; a precondition is an obligation on the trace already past; and reversing time exchanges them. Concretely, in the RE instance of §4 the right quotient is the left quotient conjugated by reversal,

$$r / s = \text{rev}(\text{rev}(r) \setminus \text{rev}(s)),$$

the same regular expression  $\Sigma^* \cdot e \cdot \Sigma^*$  serves as “ $e$  happened earlier” in a *pre* slot and “ $e$  must happen later” in a *fut* slot, and the four right-quotient axioms of §3 are the four left-quotient axioms with the direction of composition flipped. This symmetry is what makes *pre* cheap — it reuses the machinery *fut* already needs — and it also explains the one place the analogy stops: monadic bind is *oriented*, composing forward, so the backward-facing field cannot satisfy the monad laws in the same orientation. We return to this in §3.

### 2.4 How the Specification Meets Real Effects

Because annotations live inside the underlying  $\mathfrak{m}$ , the `Pledge` monad transformer integrates naturally with real effectful code, in one of two ways. When the specification primitives use `Pledge \ $ return (val, pre, post, fut)`, annotations are pure Haskell values and the underlying action does nothing; composing such primitives with  $\gg=$  builds a specification term the programmer inspects independently of running any real effect. When the underlying action is real, the annotations can instead reference actual runtime values:

```
openFile :: FilePath → IO.IOMode → Pledge IO (RE IO.Handle) IO.Handle
openFile fn mode = Pledge $ do
  h <- IO.openFile fn mode           -- actual IO
  return (h, universe,
          Single (Atom "open" h),    -- post uses the real handle h
          finally (Atom "close" h))  -- fut requires closing h specifically
```

Here  $h :: \text{IO.Handle}$  is the actual file handle produced at runtime, embedded directly into the `RE IO.Handle` annotations. The left-quotient machinery then propagates this concrete handle through

subsequent binds, so a residual `finally(close h)` names the specific unclosed handle — the mechanism that lets §5 treat `inspect` as running a monitor over a real execution rather than a specification detached from it.

### 3 What Makes Pledge Follow the Monad Law

Every monad must satisfy three equational laws. This section states, abstractly over the **Composable** algebra, exactly what suffices for **Pledge** to do so — and is equally precise about what that statement excludes. The exclusion is structural and expected: the *pre* field is oriented against the direction of bind, and is carried beside the lawful triple rather than inside it. §4 then asks, for the concrete RE instance, *which* of the axioms below actually hold, and under what condition on how annotations are written.

#### 3.1 Why *pre* sits outside the triple

Recall from §2 that *pre* and *fut* are the same notion pointing in opposite directions, and  $\backslash$  is  $\backslash$  conjugated by reversal. The monad laws break that symmetry, because bind does not respect it:  $p \gg q$  runs *p* and then *q*, so every law equates rearrangements of a forward-composed sequence. A field that accumulates forwards can satisfy such laws; a field that accumulates backwards cannot be expected to, and *pre* does not.

Concretely, left identity requires the annotations of pure  $x \gg f$  to equal those of  $f x$ . For *fut* this holds because pure contributes  $\top$  on the left, and  $\top$  is the unit that  $\backslash$  preserves (axiom L2 below). For *pre* the corresponding step needs  $\top \backslash post(f x) = \top$ , which is axiom R2 — the *mirror* of L2, and true for exactly the same reason. So *pre* is not lawless; it satisfies the mirror-image laws. What it cannot do is satisfy the forward-oriented laws simultaneously with *fut* under a single equality, because a detected precondition violation must stay visible under further binding rather than being normalised away — a Hoare-rule discipline, not an oversight.

We therefore prove the three laws for the triple (*ret*, *post*, *fut*) and carry *pre* alongside, in the manner of a writer-monad log: inspected, propagated by Equation (1), and not required to satisfy the monad laws on its own. All four fields *are* covered by the axioms below; the asymmetry is in which equations we then ask them to satisfy, not in how they are treated algebraically.

#### 3.2 Fourteen axioms

The laws hold assuming the fourteen axioms in Table 1, stated abstractly over the **Composable** algebra. The reversal duality is visible in the table: R1–R4 are L1–L4 with the order of composition reversed, and nothing else.

#### 3.3 The three laws

We verify all three laws using these axioms. In each case we show that both sides of the law produce equal (*ret*, *post*, *fut*) triples. It is worth noting which axioms each law consumes, because §4.4 shows that not all fourteen are unconditionally available for a given instance: left identity uses S1, C3, L2, R1; right identity uses S2, C1, C3, L1, R2; and associativity uses S3, C2, L3, L4, R3, R4.

*Left identity:* pure  $x \gg f = f x$ . Let  $p = \text{pure } x = (x, \top, \varepsilon, \top)$  and  $q = f x$ . By Equations (1)–(3):

$$\text{ret} : \text{ret}(q). \checkmark$$

$$\text{post} : \varepsilon \cdot \text{post}(q) = \text{post}(q). \quad (\text{S1}) \checkmark$$

$$\text{fut} : (\top \backslash \text{post}(q)) \sqcap \text{fut}(q) = \top \sqcap \text{fut}(q) = \text{fut}(q). \quad (\text{L2}, \text{C3}) \checkmark$$

For *pre*:  $\text{pre}(q) / \varepsilon = \text{pre}(q)$  (R1), so  $\text{pre} = \top \sqcap \text{pre}(q) = \text{pre}(q)$  (C3).  $\checkmark$

Table 1. The fourteen **Composable** axioms required to prove the monad laws for the  $(ret, post, fut)$  triple.

| Label | Axiom                                                               | Used for         |
|-------|---------------------------------------------------------------------|------------------|
| S1    | $\varepsilon \cdot a = a$                                           | Law 1, post      |
| S2    | $a \cdot \varepsilon = a$                                           | Law 2, post      |
| S3    | $(a \cdot b) \cdot c = a \cdot (b \cdot c)$                         | Law 3, post      |
| C1    | $a \sqcap b = b \sqcap a$                                           | Law 2, pre / fut |
| C2    | $(a \sqcap b) \sqcap c = a \sqcap (b \sqcap c)$                     | Law 3, pre / fut |
| C3    | $\top \sqcap a = a$                                                 | Laws 1, 2        |
| L1    | $r \setminus \varepsilon = r$                                       | Law 2, fut       |
| L2    | $\top \setminus r = \top$                                           | Law 1, fut       |
| L3    | $r \setminus (a \cdot b) = (r \setminus a) \setminus b$             | Law 3, fut       |
| L4    | $(a \sqcap b) \setminus c = (a \setminus c) \sqcap (b \setminus c)$ | Law 3, fut       |
| R1    | $a / \varepsilon = a$                                               | Law 1, pre       |
| R2    | $\top / r = \top$                                                   | Law 2, pre       |
| R3    | $(a / b) / c = a / (c \cdot b)$                                     | Law 3, pre       |
| R4    | $(a \sqcap b) / c = (a / c) \sqcap (b / c)$                         | Law 3, pre       |

*Right identity:*  $e \gg \text{pure} = e$ . Let  $q = \text{pure}(ret(e)) = (ret(e), \top, \varepsilon, \top)$ .

$ret : ret(e) \cdot \varepsilon$  ✓

$post : post(e) \cdot \varepsilon = post(e)$ . (S2) ✓

$fut : (fut(e) \setminus \varepsilon) \sqcap \top = fut(e) \sqcap \top = \top \sqcap fut(e) = fut(e)$ . (L1, C1, C3) ✓

*Associativity.* Let  $r_1 = ret(e)$ ,  $r_2 = ret(f r_1)$ . Both sides yield  $ret(g r_2)$  and  $post(e) \cdot post(f r_1) \cdot post(g r_2)$  (by S3). For the future, unfolding Equation (3) on the LHS and applying L4 then L3 gives:

$$((fut(e) \setminus post(f r_1)) \setminus post(g r_2)) \sqcap (fut(f r_1) \setminus post(g r_2)) \sqcap fut(g r_2)$$

which equals the RHS by C2 and L3. ✓ For *pre*, applying R4 then R3 gives the same result on both sides by C2. ✓

The three proofs above are mechanised in Lean 4 in `Formalization/PledgeMonadLaws.lean`, using an abstract **Composable** structure parametrised over the fourteen axioms of Table 1. The file carries no sorry; all proofs are term-mode derivations from the stated axioms, and depend on Mathlib [?] only for basic list and set lemmas. Because the theorem is proved once, abstractly, *any* instance that discharges the fourteen axioms inherits the monad laws for free — §4 is where we pay that debt for RE.

## 4 RE Satisfies the Monad Laws

§3 proved the monad laws *relative to* fourteen axioms. This section discharges those axioms for a concrete instance: extended regular expressions over a typed event alphabet, with Brzozowski derivatives computing both quotients. Mechanising this instance in Lean does not merely confirm the axioms — it *delimits* them, and the delimitation is the main technical content of this section.

### 4.1 Extended Regular Expressions over Typed Events

Events are the atomic units of observation. An Event  $t$  carries a name and a typed payload, or is a wildcard, or records a bare *use* of a value without naming an operation (for obligations that are about touching a resource at all, rather than any one named event on it):

```

data Event t = Atom String t -- concrete event, e.g. Atom "acquire" (Num 1)
             | Usage t       -- any access to a value, unnamed
             | Wildcard      -- matches any single event

```

The type parameter  $t$  allows the payload to be any type, including runtime values such as file handles or heap addresses; the concrete payload type used throughout the examples below, **Values**, also underlies the Presburger terms of §6. The specification language is an extended regular expression type:

```

data RE t = Bot | Epsilon | Single (Event t)
          | Seq (RE t) (RE t) | Or (RE t) (RE t)
          | And (RE t) (RE t) | Star (RE t) | Not (RE t)

```

The constructors are standard:  $\emptyset$  (**Bot**),  $\epsilon$  (**Epsilon**), single-event match (**Single**), concatenation (Seq), union (Or), intersection (And), Kleene star (**Star**), and complement (Not). And is a primitive constructor so its derivative is computed directly rather than through three extra Not nodes.

Four derived forms appear throughout the paper:

$$\begin{aligned}
\text{anything} &\triangleq \neg \emptyset && (\text{universal language, written } \top) \\
\text{finally}(e) &\triangleq \text{anything} \cdot e \cdot \text{anything} && (e \text{ must eventually occur}) \\
\text{never}(e) &\triangleq \neg \text{finally}(e) && (e \text{ must never occur}) \\
\text{noUntil}(e, g) &\triangleq \neg((\Sigma \setminus \{g\})^* \cdot e \cdot \text{anything}) && (e \text{ must not occur before } g)
\end{aligned}$$

$\text{previously}(e)$  is an alias for  $\text{finally}(e)$  used in pre slots to assert that  $e$  occurred somewhere in the preceding trace; the underlying RE is identical, but the name signals intent, and §4.6 is precisely about why this alias matters.

## 4.2 Algebraic Complement and Brzowski Derivatives

The Brzowski derivative  $[?] \partial_a(r)$  computes the language of continuations after reading event  $a$ . The key law for complement is:

$$\partial_a(\neg r) = \neg(\partial_a(r)) \quad (4)$$

Together with the De Morgan laws and double-negation elimination, this lets complement be propagated purely algebraically, without building a DFA. The nullability function  $v(r)$  ( $= \text{True}$  iff  $\varepsilon \in L(r)$ ) satisfies  $v(\neg r) = \neg v(r)$ :

```

nullable :: RE t → Bool
nullable (Not r)      = not (nullable r)
nullable (Star _)     = True
nullable (Seq r1 r2)  = nullable r1 && nullable r2
nullable (Or r1 r2)   = nullable r1 || nullable r2
nullable (And r1 r2)  = nullable r1 && nullable r2
nullable Epsilon      = True
nullable _            = False

derivative :: Eq t ⇒ Event t → RE t → RE t
derivative e (Not r)   = Not (derivative e r) -- da(Not r) = Not(da(r))
derivative e (Single p) = if subsumesEvent e p then Epsilon else Bot
derivative e (Seq r1 r2)
  | nullable r1 = Or (Seq (derivative e r1) r2) (derivative e r2)
  | otherwise   = Seq (derivative e r1) r2

```

```

derivative e (Or r1 r2) = Or (derivative e r1) (derivative e r2)
derivative e (And r1 r2) = And (derivative e r1) (derivative e r2)
derivative e (Star r)    = Seq (derivative e r) (Star r)
derivative _ _          = Bot

```

Complement requires care when computing which events can begin a word: naively returning  $first(\neg r, \Sigma_0) = first(r, \Sigma_0)$  is unsound. The correct characterisation, over a finite working alphabet  $\Sigma_0$ , is

$$e \in first(\neg r, \Sigma_0) \iff e \in \Sigma_0 \wedge [\partial_e(r)] \neq \neg\emptyset, \quad (5)$$

that is,  $e$  can open a word in  $L(\neg r)$  iff, after consuming  $e$ , the residual  $\partial_e(r)$  does not cover the entire language. The working alphabet itself must include **Wildcard** as the representative of “some event other than the named ones”; omitting it is not a conservative simplification but a soundness bug, since a pair of operands naming no concrete event at all (such as  $\Sigma^*$ ) would then have an empty working alphabet and quotient to  $\emptyset$  regardless of its actual denotation.

### 4.3 The Composable RE Instance

```

instance Eq t => Composable (RE t) where
  concatenation = Seq
  conjunction  = And
  empty        = Epsilon
  universe     = Not Bot      -- = Sigma*
  leftQuotient = reLeftQuotient
  rightQuotient = reRightQuotient

```

The left-quotient  $r_2 \setminus r_1$  is computed by a worklist traversal over reachable pairs of Brzozowski derivatives, which keeps termination tractable and is what `reLeftQuotient` implements. The right-quotient is computed via reversal,  $r_2 / r_1 = rev(rev(r_2) \setminus rev(r_1))$ , where  $rev(r)$  reverses the sequences accepted by  $r$  – the mechanical form of the reversal duality of §2. A normaliser applies the laws in Table 2 to simplify RE terms, including  $\Sigma^* \cdot r = \Sigma^*$  for nullable  $r$ , without which a fully-discharged residual such as  $\Sigma^* \cdot (a \cdot \Sigma^* \vee \varepsilon)$  would stay syntactically distinct from  $\top$  and be reported as outstanding.

Table 2. Normalisation laws applied by `normalize`, recursively bottom-up.

| Law                                                                                     | Constructor |
|-----------------------------------------------------------------------------------------|-------------|
| $\varepsilon \cdot r = r, r \cdot \varepsilon = r, \top \cdot r = \top$ ( $r$ nullable) | Seq         |
| $\emptyset \vee r = r, r \vee \top = \top$                                              | Or          |
| $\emptyset \wedge r = \emptyset, r \wedge \top = r$                                     | And         |
| $\neg \neg r = r$ , De Morgan, $\neg \emptyset = \top, \neg \top = \emptyset$           | Not         |
| $\emptyset^* = \varepsilon, \varepsilon^* = \varepsilon$                                | Star        |

*Termination.* The quotient recursion terminates because the set of distinct normalised derivatives of any regular expression is finite modulo associativity, commutativity and idempotence of `Or`/`And` – a classical result of Brzozowski [?], extended to complement by Owens et al. [?]. Cycle detection in the worklist must therefore compare states modulo those identities, not structurally: a divisor whose derivative does not shrink under structural comparison, such as  $\Sigma^*$ , would otherwise make the recursion diverge.

#### 4.4 What the RE instance actually satisfies

Ten of the fourteen axioms hold outright for RE; four do not unconditionally, and Table 3 records which.

Table 3. Status of the fourteen axioms for the RE instance, as mechanised in Formalization/REComposableLaws.lean.

| Axioms         | Status             | Condition                            |
|----------------|--------------------|--------------------------------------|
| S1–S3, C1–C3   | hold               | —                                    |
| L1, L3, R1, R3 | hold               | —                                    |
| L2, R2         | hold conditionally | $L(\text{divisor}) \neq \emptyset$   |
| L4, R4         | one inclusion only | $\subseteq$ holds; $\supseteq$ fails |

*Why L2 and R2 need a non-empty divisor.*  $\top \setminus r$  collects every suffix left over after consuming some word of  $L(r)$ . If  $L(r) = \emptyset$  there is no such word and the quotient is  $\emptyset$ , not  $\top$ : a postcondition denoting the empty language would assert that the action produced an impossible trace.

*Why L4 and R4 hold in only one direction.* Written out, L4 asks  $(a \sqcap b) \setminus c = (a \setminus c) \sqcap (b \setminus c)$ . The left-hand side demands *one* word  $u \in L(c)$  that works for both conjuncts; the right-hand side lets each conjunct choose its own. Taking  $L(c) = \{a, b\}$ ,  $L(a) = \{a\}$  and  $L(b) = \{b\}$ : on the left,  $L(a) \cap L(b) = \emptyset$ , so the quotient is  $\emptyset$ ; on the right, the first conjunct picks  $u = a$  and the second picks  $u = b$ , each leaving  $\varepsilon$ , so the meet is  $\{\varepsilon\}$ . Only  $\subseteq$  survives.

*Consequence: associativity is an inclusion.* Associativity consumes L4 and R4, so in general it holds only as

$$\text{fut}((p \ggg f) \ggg g) \subseteq \text{fut}(p \ggg (\lambda x. f \ x \ggg g)),$$

and symmetrically for *pre*, the left-nested bracketing yielding the *stronger* obligation. This is not a vacuous gap: over a pool of nine representative annotations in each of five positions, the two bracketings differ in 902 of 59,049 combinations, strictly in the direction stated every time.

#### 4.5 Single-word postconditions restore equality

The gap closes under a condition that costs nothing in practice.

LEMMA 4.1 (UNIQUE WITNESS). *If  $L(c)$  is a single word  $\{u\}$ , then L4 and R4 hold with equality, and L2 and R2 hold.*

PROOF.  $\{u\}$  is non-empty, giving L2 and R2. For L4,  $(a \sqcap b) \setminus \{u\} = \{w \mid uw \in L(a) \cap L(b)\} = \{w \mid uw \in L(a)\} \cap \{w \mid uw \in L(b)\} = (a \setminus \{u\}) \sqcap (b \setminus \{u\})$ ; the choice of witness is forced, so the two sides cannot diverge. R4 is the mirror image.  $\square$

THEOREM 4.2 (LAWS FOR SINGLE-WORD POSTCONDITIONS). *If every primitive's post denotes a single word, then all fourteen axioms hold for the RE instance and the three monad laws hold with equality.*

PROOF. Every  $\setminus$  and  $/$  in Equations (1) and (3) has a *post* field as its divisor, so Lemma 4.1 applies at every use; the remaining ten axioms hold unconditionally.  $\square$

The hypothesis is not a restriction, because it is what the library produces anyway and is preserved by composition: a primitive emits exactly one event; pure and **liftPledge** emit  $\varepsilon$ ; and by Equation (2) the postcondition of a composite is  $\text{post}(p) \cdot \text{post}(q)$ , a concatenation of two single-word languages and hence itself a single word. What the condition rules out is a primitive that

advertises a *choice* of postconditions — Or, **Star** or a complement in a *post* slot; such an annotation is expressible and silently costs associativity, which §8 returns to as an API-hardening opportunity.

#### 4.6 Preconditions must describe the whole past

A second side condition governs *pre*, and violating it is the more insidious failure of the two, because it produces a confident report of a contract violation for a correct program.

The natural way to annotate release is “an acquire must have happened”, and the natural way to write that is  $pre = \{\text{acquire}\}$  — the singleton language. That is wrong, and not for the reason one expects: it says the acquire must be the *immediately preceding* event, and such a condition does not survive composition. Unfolding Equation (1) for  $p \gg q$  with  $pre(q) = \{e\}$  and  $post(p) = \{e\}$ :

$$pre(q) / post(p) = \{w \mid w \cdot e \in \{e\}\} = \{\varepsilon\},$$

so  $pre(p \gg q) = pre(p) \sqcap \{\varepsilon\}$ , which is  $\emptyset$  whenever  $pre(p)$  does not accept the empty trace. The obligation was discharged, and the residual nevertheless reports a violation.

The fix is to require preconditions to be closed under extension on the left — properties of the entire preceding trace, not of its last event.

*Definition 4.3 (Past-closed).* A language  $P$  is *past-closed* if  $P = \Sigma^* \cdot P$ : whenever  $w \in P$  and  $u \in \Sigma^*$ , also  $uw \in P$ .

**PROPOSITION 4.4 (CLOSURE).** *Past-closed languages contain  $\top$  and are closed under  $\sqcap$  and under / by an arbitrary divisor. Consequently, if every primitive’s *pre* is past-closed, so is the *pre* of every program composed from them.*

**PROOF.**  $\top = \Sigma^*$  is past-closed. For  $\sqcap$ :  $\Sigma^*(A \sqcap B) \subseteq \Sigma^*A \sqcap \Sigma^*B = A \sqcap B$ , and the reverse inclusion takes  $u = \varepsilon$ . For  $/$ : let  $w \in P / s$ , so  $wu \in P$  for some  $u \in L(s)$ ; for any  $v$ , past-closure of  $P$  gives  $vwu \in P$ , hence  $vw \in P / s$ . The final claim follows by induction on program structure, since Equation (1) builds *pre* from  $\sqcap$  and  $/$  alone.  $\square$

The previously( $e$ ) =  $\Sigma^* \cdot e \cdot \Sigma^*$  combinator of §4.1 is past-closed, and is the intended way to write a precondition; the singleton  $\{e\}$  is not. With it, the collapse above does not occur: previously( $e$ ) /  $\{e\} = \Sigma^*$ , the obligation is discharged to  $\top$ , and the enclosing  $\sqcap$  leaves  $pre(p)$  untouched. Proposition 4.4 guarantees this is stable under further composition.

Neither this condition nor Theorem 4.2 is enforced by the type *eff*, and both are easy to violate by writing the annotation that first comes to mind; §8 discusses making them unwriteable rather than merely documented.

#### 4.7 Mechanisation and validation

A companion Lean file, Formalization/REComposableLaws.lean, develops the RE instance at the level of languages: it proves the ten unconditional axioms, proves L2 and R2 under the non-emptiness hypothesis, proves the  $\subseteq$  direction of L4 and R4 and exhibits the counterexample to the converse, and proves derivative and nullable correctness. It carries no sorry.

We complement the mechanisation with exhaustive testing over bounded pools, which is what first exposed the boundary. Quotients are checked against their set-theoretic definition  $w \in r \setminus s \iff \exists u \in L(s). uw \in L(r)$  over all pairs from a pool of 90 regular expressions and all words up to length two — 56,700 checks for each quotient, with no discrepancy. The axioms are then checked over the same pool, reproducing Table 3 exactly and independently of the Lean development. Finally, restricting postconditions to single words as Theorem 4.2 requires, and letting futures and preconditions range over RE terms including complement, star and intersection,

associativity holds in all 43,200 combinations tested — against the 902 failures found without the restriction.

#### 4.8 Soundness

A fully-discharged residual ( $\neg\emptyset$  in *fut*,  $\neg\emptyset$  in *pre*) serves as a certificate of contract-correctness. We now prove that this certificate is sound. For an RE  $r$  and a finite trace  $w$ , write  $w \models r$  for  $w \in L(r)$  and  $\partial_w(r)$  for the iterated derivative.

LEMMA 4.5 (DERIVATIVE CORRECTNESS). *For all  $r$  and  $w$ :  $w \models r$  iff  $v(\partial_w(r)) = \text{True}$ .*

PROOF. By induction on  $w$ , the base case being the definition of  $v$ , and the step using derivative correctness for regular languages [?], extended to Not by Equation (4) and to And by the direct derivative clause.  $\square$

Throughout this subsection we assume the single-word condition of Theorem 4.2, without which §4.4 showed that associativity holds only as an inclusion, so “the residual of  $e$ ” would not be determined by  $e$  alone and there would be nothing well-defined to prove sound. Under the assumption, a program  $e$  built from primitives  $p_1, \dots, p_n$  by pure and  $\gg$  emits one determinate trace  $w = u_1 \cdots u_n$ , where  $\text{post}(p_i) = \{u_i\}$ ; write  $w_{<i}$  and  $w_{>i}$  for the trace before and after  $p_i$  runs.

THEOREM 4.6 (SOUNDNESS OF RE-RESIDUAL CHECKING). *Let  $e$  be a **Pledge**  $m$  (RE  $t$ ) a term built from pure and  $\gg$  over primitives  $p_1, \dots, p_n$  whose postconditions each denote a single word. If  $L(\text{fut}(e)) = \Sigma^*$  and  $L(\text{pre}(e)) = \Sigma^*$ , then for every  $i$ :*

$$w_{>i} \models \text{fut}(p_i) \quad \text{and} \quad w_{<i} \models \text{pre}(p_i).$$

PROOF. By induction on  $n$ ; the base case  $n = 0$  is vacuous. For the step write  $e = p \gg f$  with  $p = p_1$  and  $f$  yielding  $e'$ , built from  $p_2, \dots, p_n$ , noting  $\text{post}(e') = \{w_{>1}\}$ .

*Futures.* By Equation (3),  $\text{fut}(e) = (\text{fut}(p) \setminus \text{post}(e')) \sqcap \text{fut}(e')$ ; since  $\sqcap$  is intersection and  $L(\text{fut}(e)) = \Sigma^*$ , both conjuncts denote  $\Sigma^*$ . The first gives  $L(\text{fut}(p) \setminus \{w_{>1}\}) = \Sigma^*$ , i.e.  $w_{>1} v \models \text{fut}(p)$  for every  $v$ ; taking  $v = \varepsilon$  gives the claim for  $i = 1$ . The second gives  $L(\text{fut}(e')) = \Sigma^*$ , so the induction hypothesis applies to  $e'$ , giving the claim for  $i \geq 2$ .

*Preconditions.* Symmetrically,  $\text{pre}(e) = \text{pre}(p) \sqcap (\text{pre}(e') / \text{post}(p))$ , so both conjuncts denote  $\Sigma^*$ ; the first makes  $w_{<1} = \varepsilon$  satisfy  $\text{pre}(p)$ , and the second, with  $v = \varepsilon$ , makes  $u_1$  a trace establishing  $\text{pre}(e')$ , so the induction hypothesis on  $e'$  gives the claim for  $i \geq 2$ .  $\square$

*It is a soundness result only:* a discharged residual certifies that the obligations are met, but a non-discharged residual does not by itself certify that they are not, which is exactly what §4.6’s past-closure condition protects. *The implementation tests a sufficient syntactic condition:* the theorem is stated semantically ( $L(\cdot) = \Sigma^*$ ), while **Pledge** computes  $\lceil \cdot \rceil$ , a normaliser, and reports discharge when the result is syntactically  $\neg\emptyset$  — sound but incomplete for language universality, so the check can fail to recognise a discharged residual, never the reverse. Theorem 4.6 is the foundation both front-ends built in §5–§7 inherit unchanged, since both compile to RE rather than reimplementing quotient or derivative.

*A residual scope note.* The Event type of §4.1 still has no pattern between an exact **Atom** match and the catch-all **Wildcard** — there is no way to say “a write with any arguments,” the first thing a specification author reaches for. Adding a name-indexed wildcard would remove the restriction without disturbing the algebra above: only `subsumesEvent` and the derivative’s **Single** clause depend on the exact-match reading. We leave it as a small, self-contained item, not a limitation the theory above depends on.

## 5 Runtime Monitoring with LTL, on Top of RE

§2.4 showed that a **Pledge** annotation can reference a real runtime value, and §4.8 proved that its residual is sound with respect to the trace an execution actually emits. Put together, these two facts license a reading of **Pledge** that is stronger than “a specification”: because **bind** runs the first action before computing the second, the residual after each step is the state of a monitor over the one execution that happened, and the quotient is the monitor’s transition function. This section makes that reading concrete with a proper linear-temporal-logic front-end, **Pledge.LTL**, whose formulas compile to the RE of §4 and whose monad laws and soundness theorem it therefore inherits without further proof.

### 5.1 The LTL Type

LTL over finite traces [?] is the natural logic for the obligations of §1: a mutex is released, or a handle closed, within the run, not somewhere along an infinite one.

```
data LTL t
  = LTLTrue | LTLFalse | LTLAtom (Event t) | LTLNot (LTL t)
  | LTLAnd (LTL t) (LTL t) | LTLOr (LTL t) (LTL t)
  | LTLNext (LTL t)           -- X phi
  | LTLUntil (LTL t) (LTL t) -- phi U psi
  | LTLFinally (LTL t)       -- F phi
  | LTLGlobally (LTL t)      -- G phi
  | LTLFromRE (RE t)         -- escape hatch: an opaque RE
```

Its standard translation goes through an automaton, but because complement is a first-class RE constructor (§4.1), every LTL operator translates directly to RE with no automaton construction, via **ltlToRe** (Table 4); a best-effort inverse **reToLtl** recognises the RE shapes **ltlToRe** produces and is used only for pretty-printing.

Table 4. LTL to RE translation ( $\Sigma^1 \triangleq \text{Single Wildcard}$ ,  $\neg\emptyset \triangleq \neg\emptyset$ ).

| LTL formula        | RE $\llbracket \cdot \rrbracket$                                        |
|--------------------|-------------------------------------------------------------------------|
| $\top$             | $\neg\emptyset$                                                         |
| $\perp$            | $\emptyset$                                                             |
| $e$                | $\{e\}$                                                                 |
| $\neg\phi$         | $\neg\llbracket \phi \rrbracket$                                        |
| $\phi \wedge \psi$ | $\llbracket \phi \rrbracket \cap \llbracket \psi \rrbracket$            |
| $\phi \vee \psi$   | $\llbracket \phi \rrbracket \cup \llbracket \psi \rrbracket$            |
| $X\phi$            | $\Sigma^1 \cdot \llbracket \phi \rrbracket$                             |
| $F\phi$            | $\neg\emptyset \cdot \llbracket \phi \rrbracket$                        |
| $G\phi$            | $\neg(\neg\emptyset \cdot \neg\llbracket \phi \rrbracket)$              |
| $\phi U \psi$      | $step(\phi)^* \cdot \llbracket \psi \rrbracket$ ( $\phi$ propositional) |

The **Composable LTL** instance routes every operation through **ltlToRe**, wrapping the RE result back up as **LTLFromRE**, so **concatenation**, the two quotients, and the two identities are exactly RE’s, viewed through the LTL syntax:

```
instance Eq t => Composable (LTL t) where
  concatenation l1 l2 = LTLFromRE (concatenation (ltlToRe l1) (ltlToRe l2))
  conjunction      = LTLAnd
```

```

empty          = LTLFromRE empty
universe       = LTLTrue
leftQuotient   l1 l2 = LTLFromRE (leftQuotient (l1ToRe l1) (l1ToRe l2))
rightQuotient  l1 l2 = LTLFromRE (rightQuotient (l1ToRe l1) (l1ToRe l2))

```

## 5.2 Monitoring a Mutex and a Transaction

We read two of the scenarios from §1 as monitor runs: each primitive posts an event and a future obligation, `inspect` executes the program exactly once, and the residual reports the monitor’s state at the end of that one execution.

```

acquire :: Int → Pledge IO (RE Values) ()
acquire mid = Pledge $ return
  ( ()
  , universe
  , Single (Atom "acquire" (List [Num mid]))
  , finally (Atom "release" (List [Num mid])) )

release :: Int → Pledge IO (RE Values) ()
release mid = Pledge $ return
  ( ()
  , previously (Atom "acquire" (List [Num mid])) -- whole-past precondition
  , Single (Atom "release" (List [Num mid]))
  , universe )

```

The precondition uses `previously`, not `Single`: by §4.6 it must describe the whole preceding trace, or nestedLocks below — where `release 2` intervenes between the `acquire` and its matching `release` — would report a violation for a correct run.

| Monitored run                                               | Residual <i>fut</i>              | Residual <i>pre</i>              |
|-------------------------------------------------------------|----------------------------------|----------------------------------|
| <code>do acquire 1; release 1</code>                        | $\neg\emptyset$                  | $\neg\emptyset$                  |
| <code>do acquire 1; acquire 2; release 2; release 1</code>  | $\neg\emptyset$                  | $\neg\emptyset$                  |
| <code>do acquire 1; acquire 2; release 2 (lock leak)</code> | <code>finally(release(1))</code> | $\neg\emptyset$                  |
| <code>release 1 (no matching acquire)</code>                | $\neg\emptyset$                  | <code>finally(acquire(1))</code> |

The lock-leak run’s residual future names lock 1 specifically: lock 2’s obligation is discharged by `release 2`’s postcondition, and lock 1’s `finally(release(1))` passes through the left quotient unchanged. The unmatched `release 1`’s residual precondition `finally(acquire(1))` reads as an obligation transferred to the caller: composing this fragment after `acquire 1` discharges it to  $\neg\emptyset$ . This is the case a lexically scoped combinator such as `bracket` handles least well: two obligations are outstanding when `release 2` runs, and which one it discharges is not determined by the program’s nesting, only by which event actually occurred — exactly what a monitor, and not a static scope, can see.

A disjunctive obligation makes the same point from the other direction. `beginTx` posts a future requiring eventual commit or rollback; whichever event the run actually emits discharges the whole disjunction via the left quotient, and `beginTx` alone retains it in full. No pre/post pair can express “resolved by either of two later events, one of which is chosen at runtime”; a monitor whose state is the residual itself can, because that state is exactly a set of continuations, not a Boolean flag.

## 6 Static Verification with GuardedRE, on Top of RE

RE and its LTL front-end are silent about arithmetic state, and their soundness theorem (§4.8) is inherently about *one* executed trace: checking it is runtime monitoring. The GRE instance adds a second, independent dimension — a Presburger arithmetic predicate over heap addresses, discharged by Z3 via sbv — and because that predicate is checked over *symbolic* heap values rather than the concrete heap of one run, the same query can be asked *before* any execution: is this residual guard satisfiable at all, for some heap; is it valid for every heap satisfying an invariant. That is what makes GRE a static-verification instance rather than a second runtime monitor.

### 6.1 The GuardedRE Type

```
type GuardedRE a = [(Pred, RE a)]
```

A state (heap, trace) satisfies `gre :: GuardedRE a` iff *some* disjunct  $(p, r) \in gre$  has  $\text{heap} \models p$  and  $\text{trace} \in L(r)$ . Within one disjunct the two dimensions are independent — the predicate is static and the RE is advanced by derivatives — but the list carries *disjunction* across dimensions: a plain conjunctive pair  $(p, r)$  cannot express “the heap is in state  $A$  and the trace obeys  $r_A$ , or the heap is in state  $B$  and the trace obeys  $r_B$ ”, which is exactly the shape a property that changes as the heap evolves takes, and is what an earlier, single-conjunct design could not express. Smart constructors lift a pure RE or a pure predicate:

```
fromRE    :: RE a  → GuardedRE a
fromPPred :: Pred  → GuardedRE a
```

### 6.2 The Composable GuardedRE Instance

```
instance Eq a ⇒ Composable (GuardedRE a) where
  concatenation xs ys = [ (normalizePred (PAnd p1 p2), normalize (Seq r1 r2))
                        | (p1,r1) <- xs, (p2,r2) <- ys ]
  conjunction  xs ys = [ (normalizePred (PAnd p1 p2), normalize (And r1 r2))
                        | (p1,r1) <- xs, (p2,r2) <- ys ]
  empty        = [(PTrue, Epsilon)]
  universe     = [(PTrue, top)]
  leftQuotient xs ys = [ (normalizePred (PAnd p1 p2), normalize (reLeftQuotient r1
                                r2))
                        | (p1,r1) <- xs, (p2,r2) <- ys ]
  rightQuotient xs ys = [ (normalizePred (PAnd p1 p2), normalize (reRightQuotient r1
                                r2))
                        | (p1,r1) <- xs, (p2,r2) <- ys ]
```

Every operation is the cross product of disjuncts, conjoining the Pred halves and applying the corresponding RE operation to the trace halves; a normaliser (`normalizeGuarded`) drops unsatisfiable-looking disjuncts and merges disjuncts that share a predicate, keeping the list from growing unboundedly under repeated composition, and an SMT-backed variant (`normalizeGuardedSMT`) additionally collapses disjuncts whose predicates are jointly exhaustive — e.g. a two-way heap split  $h[a] = 0 \vee h[a] > 0$  collapses to a single `PTrue` disjunct — which needs a solver call to see and so lives in IO.

### 6.3 Presburger Predicates, and Two Kinds of Solver Query

```

data Term = Val Values | Add Term Term | Neg Term | Mul Int Term
data Pred = PTrue | PFalse
          | PLt Term Term | PLe Term Term | PEq Term Term
          | PGt Term Term | PGe Term Term
          | PNot Pred | PAnd Pred Pred | POr Pred Pred

```

Val (**ValAt** a) reads the value at heap address a. Two solver queries give the static reading its two flavours: `checkPPred :: Pred → IO SolverResult` asks Z3 for a witness heap satisfying a predicate (an existential, SAT query over free heap and value variables), and `isValidUnderHeapInvariant :: Pred → IO Bool` asks whether a predicate holds for *every* heap obeying the standing invariant that addresses are non-negative, by checking that its negation, conjoined with that invariant, is unsatisfiable. Neither call needs a concrete run: both are posed over the symbolic addresses the predicate mentions. `checkGuarded` folds `deriveGuarded` (which advances only the RE half of every disjunct) over a trace and then asks `checkPPred` on the nullable disjuncts against a *concrete* heap, giving GRE a runtime reading as well when one is wanted — the two modes share the same representation and differ only in whether the predicate side is instantiated or left symbolic.

## 6.4 Heap Liveness and Allocation Order

Manual memory management needs a trace constraint (malloc must precede its matching free, which must eventually happen, and a second free must not occur before re-allocation) and a heap constraint (the cell is live,  $h[a] > 0$ ) at once; an RE alone cannot see liveness, and a predicate alone cannot see ordering. Each malloc/free pair contributes *two* disjuncts, one per heap state the cell can be in, so the obligation tracks a property that changes as the trace proceeds rather than only a static invariant:

```

malloc :: Pledge IO (GuardedRE Values) Addr
malloc = Pledge $ do
  addr <- randomRIO (0, 5)
  return ( addr
          , fromRE universe
          , [ (PEq (Val (ValAt addr)) (Val (Num 0))), Epsilon)
            , (PGt (Val (ValAt addr)) (Val (Num 0))), Single (Atom "malloc" (List
[Num addr]))) ]
          , [ (PEq (Val (ValAt addr)) (Val (Num 0))), never (Usage (List [Num
addr])))]
          , (PGt (Val (ValAt addr)) (Val (Num 0))), finally (Atom "free" (List
[Num addr])))] ]
    )

free :: Addr → Pledge IO (GuardedRE Values) ()
free addr = Pledge $ return
  ( ()
  , [ (PEq (Val (ValAt addr)) (Val (Num 0))), universe)
    , (PGt (Val (ValAt addr)) (Val (Num 0))), previously (Atom "malloc" (List [Num
addr])))] ]
  , fromRE (Single (Atom "free" (List [Num addr])))
  , fromRE (noUntil (Atom "free" (List [Num addr])) (Atom "malloc" (List [Num
addr])))] )

```

| Program                           | Residual <i>pre</i>                              | Residual <i>fut</i>                                |
|-----------------------------------|--------------------------------------------------|----------------------------------------------------|
| mallocThenFree, reallocate (good) | $[h=0] \neg \emptyset \vee [h>0] \neg \emptyset$ | $[h>0] \text{noUntil}(\text{free}, \text{malloc})$ |
| missingFree (leak)                | $[\text{true}] \neg \emptyset$                   | $[h>0] \text{finally}(\text{free})$                |
| freeWithoutMalloc                 | $[h>0] \text{finally}(\text{malloc})$            | —                                                  |
| doubleFree                        | $[h=0] \neg \emptyset \vee [h>0] \neg \emptyset$ | $\emptyset$                                        |

doubleFree reaches an empty future because the second free must satisfy the noUntil guard the first one imposed, and no trace does; reallocate shows the same guard reset cleanly by an intervening malloc. Between  $\neg \emptyset$  and  $\emptyset$  lies a satisfiable-but-not-universal residual, which reads as a standing constraint on whatever runs next rather than a verdict — mallocThenFree is correct and still carries noUntil(free, malloc) forward, precisely because that obligation belongs to the continuation, not to the pair itself.

## 7 Checking Out MTL, on Top of RE

LTL (§5) is qualitative about time:  $F \phi$  says  $\phi$  happens *eventually*, with no bound on when. Real protocols are often quantitative instead — a lock must be released *within five steps*, a valve must close *between two and four steps* after opening. **Pledge.MTL** adds bounded, discrete-step intervals to the same syntax, and — like LTL — compiles down to RE rather than adding a new algebra, so it inherits §4’s monad laws and soundness theorem for free.

### 7.1 Bounded Intervals and the MTL Type

```
data Interval = Interval Int (Maybe Int)  -- [lo, hi] or [lo, infinity)
```

```
within, exactly, atLeast :: Int → Interval
```

```
fromTo :: Int → Int → Interval
```

```
data MTL t
= MTLTrue | MTLFalse | MTLAtom (Event t) | MTLNot (MTL t)
| MTLAnd (MTL t) (MTL t) | MTLOr (MTL t) (MTL t) | MTLNext (MTL t)
| MTLUntil Interval (MTL t) (MTL t)  -- phi U_[lo,hi] psi
| MTLFinally Interval (MTL t)         -- F_[lo,hi] phi
| MTLGlobally Interval (MTL t)        -- G_[lo,hi] phi
| MTLFromRE (RE t)
```

A bounded modality compiles by finite unrolling:  $F_{[lo,hi]} \phi$  becomes a union, over each step count  $k \in [lo, hi]$ , of “skip exactly  $k$  events, then  $\phi$ ”:

```
mtlToRe (MTLFinally i p) = do
  ks <- intervalSteps i
  r <- mtlToRe p
  pure $ case ks of
    Left a → Seq (sigmaPow a) (Seq top r)  -- unbounded above: [a,
    infinity)
    Right xs → foldr Or Bot [ Seq (sigmaPow k) r | k <- xs ]  -- [lo, hi]
```

where sigmaPow  $n = \text{Seq} \text{ (Single Wildcard) } (\text{sigmaPow } (n-1))$  concatenates exactly  $n$  wildcards.  $G_{[lo,hi]} \phi$  and  $\phi U_{[lo,hi]} \psi$  unroll the same way, using atMostSeq and reptSeq respectively to bound the number of repetitions; a best-effort inverse reToMtl mirrors §5’s reToLtl for pretty-printing. The **Composable MTL** instance is the same shape as **Composable LTL**: every operation round-trips

through `mtlToRe` and wraps the RE result as `MTLFromRE`, so an MTL specification is, after compilation, an ordinary RE specification that §4 already knows how to discharge.

## 7.2 A Bounded Obligation

Three combinators cover the common patterns directly, without spelling out an `Interval` at each use site:

```
finallyWithin :: Int → Event t → MTL t -- F_[0,n] ev
neverWithin   :: Int → Event t → MTL t -- not F_[0,n-1] ev
globallyWithin :: Int → Event t → MTL t -- G_[0,n-1] ev
```

Annotating a valve's open with `finallyWithin 5 (Atom "close" v)` as its future obligation reads, after compilation, as a five-way union of “close occurs after exactly  $k$  other events” for  $k \in [0, 5]$ ; a run that closes the valve on step 3 discharges it via the same left-quotient machinery as an unbounded `finally`, and a run that never closes it leaves the same kind of  $\text{non-}\neg\emptyset$  residual as §5.2's lock leak, now additionally carrying the deadline that was missed. A worked case study exercising MTL end to end, alongside the §5.2 and §6.4 examples, is not yet in the Examples tree; adding one is a short, self-contained task we list under §10.

## 8 Usability: Making Error Location Traceable

Everything up to this point reports a violation as a value: a  $\text{non-}\neg\emptyset$  *fut* or a collapsed *pre*. Nothing yet says *where* in a composed program that value first stopped being  $\neg\emptyset$ . For a two-line example this does not matter; for the kind of program `Pledge` is meant to specify, it does, so we record this as a concrete, currently unimplemented piece of the design rather than fold it into the discussion.

*The problem.* `inspect` runs the whole composed action and reports only the final residual; a violation folded in at bind step 3 of 30 looks identical to one folded in at step 29. The programmer must bisect the `do`-block by hand to find the offending primitive.

*Plan.*

- Add a labelling combinator, `label :: String → Pledge m eff a → Pledge m eff a` (or a `HasCallStack` variant that needs no explicit string), that tags one primitive's call site without changing its annotations.
- Thread an accumulating trace of  $(\text{label}, \text{pre}, \text{post}, \text{fut})$  triples through `bind`, alongside the existing 4-tuple, so a labelled step's contribution to the residual is individually recoverable rather than only visible in the folded-in result.
- Report, on inspection, the *first* labelled step at which *pre* became  $\emptyset$  or the running *fut* stopped being  $\neg\emptyset$  — turning “the program violates its contract” into “line  $N$  does”.
- Make this opt-in and free when unused: a program that never calls `label` pays no overhead beyond what §2 already costs.

This closes a gap independently noted in early user feedback on the library (violations are silent and unlocated) and pairs naturally with a second, related item: nothing currently stops execution when *pre* collapses to  $\emptyset$ , so a `runChecked` that inspects the residual after each `bind` and fails as soon as it reaches  $\emptyset$  would turn a located violation into an enforced one, at the cost of a quotient computation per `bind`. Both are additions to `Pledge.Core`; neither requires revisiting §3–§7.

## 9 Auto-Generation of Proofs

§3 proves the monad laws once, abstractly, from fourteen axioms; §4.7 then discharges those fourteen axioms for RE by hand in Lean. Every new `Composable` instance — GRE today, whatever comes after it tomorrow — currently repeats that second step from scratch: a person mirrors the

Haskell definition in Lean and proves fourteen lemmas again. This does not scale past a handful of instances, and it is the piece of the mechanisation story we have not automated.

*Plan.*

- **Property-test first.** Before writing any Lean, state the fourteen axioms as QuickCheck properties over the candidate instance — exactly what §4.7 already does for RE — and run them over a bounded pool. A property that fails here needn't be attempted in Lean; §4.4 is what a systematic sweep of this kind found for RE.

- **Generate the Lean statements, not the proofs, mechanically.** The fourteen axiom statements are the same schema for every instance, parametrised only by the instance's **concatenation**, **conjunction**, and quotients; a small script can emit the Lean lemma *signatures* for a new instance directly from its Haskell type-class instance, leaving only the proof bodies to fill in.

- **Discharge the finite fragment automatically.** Several axioms (S1–S3, C1–C3 in Table 1) are generic properties of an underlying monoid/semilattice pair and are provable by `decide` or a Mathlib tactic once the instance's carrier is finite or the operations are given as pattern matches, without a hand-written proof term; only L2–L4 and R2–R4 need instance-specific reasoning about the quotient.

- **Track the boundary as data.** Record, per instance, exactly which axioms hold unconditionally, which hold under a stated side condition, and which fail — as Table 3 does for RE — so that adding an instance produces a table entry automatically alongside whatever manual proof the residual cases still need.

The target is not a fully automatic prover for arbitrary instances, which is out of scope; it is removing the boilerplate that is already mechanical today, so that mechanising a fifth instance costs a fraction of what the first two did.

## 10 Experiments

The evaluation so far is entirely about correctness — the property-based validation of §4.7 and the case studies of §5.2 and §6.4 — and says nothing about cost. That evaluation is not yet done; we scope it concretely rather than claim results we do not have.

*Plan.*

- **Quotient and residual-size growth.** For the RE case studies of §5.2, measure derivative/quotient wall-clock time and normalised-residual size as a function of trace length, to check that the worklist termination argument of §4.3 stays cheap in practice and not only in the worst case.

- **SMT call cost for GRE.** For §6.4, count and time the `checkPPred / isValidUnderHeapInvariant` calls per program, and report how much `normalizeGuardedSMT` reduces disjunct-list growth compared to the purely structural `normalizeGuarded`.

- **Bounded vs. unbounded compilation.** Compare MTL (§7) against LTL (§5) on the same obligation as the interval width grows, to characterise the unrolling blow-up `sigmaPow/reptSeq` introduce and find where it stops being negligible.

- **Scale up the axiom validation.** Extend §4.7's bounded-pool QuickCheck sweep with larger pools and longer words, and add the GRE instance to it, both as a correctness check and as the first input to §9's property-test-first step.

- **Report, don't assert.** Present the above as tables/plots over the existing Examples programs plus the one new MTL case study noted in §7, rather than a synthetic benchmark suite, so the numbers describe the same programs the rest of the paper already discusses.

## 11 Related Work

*Future conditions and temporal verification.* The proposal this paper implements is due to ?, who introduce future conditions as a first-class extension to pre/post contracts at the level of a formal system. ? apply a related RE encoding of temporal properties in ProveNFix to guide automated program repair. Pledge’s contribution is a concrete, runnable library realisation: a monad transformer, a derivative-based discharge mechanism, and a mechanised account of exactly when it is lawful, neither prior paper gives.

*Pre/post-condition contracts in Haskell.* Liquid Haskell [?] expresses pre/post-conditions as refinement types verified by an SMT solver. Findler and Felleisen [?] establish runtime contract discipline for higher-order functions. Pledge complements these by adding a temporal future condition without requiring type-system extensions.

*Specifications as monads.* The idea that a specification can itself be monadic structure is the closest conceptual neighbour to this work. Hoare Type Theory [?] indexes a monad by a pre/post pair, making Hoare triples types. The *Dijkstra monad* [?] instead indexes computations by predicate transformers, so that the weakest precondition of a composite is computed by bind; ? derive such monads mechanically from an effect’s definition, and ? give the general account in which a Dijkstra monad is a monad morphism into a specification monad. ? develop the predicate-transformer view for effects in a dependently typed setting.

The shared idea is that specifications compose the way programs do; the difference is what composition computes. A Dijkstra monad propagates *backwards*: bind computes the weakest precondition of a sequence from its continuation’s, and the result is a predicate transformer to be discharged by a verifier. **Pledge** propagates *forwards* as well, and its *fut* field has no counterpart in that line of work — a predicate transformer relates two states, whereas a future condition constrains the trace still to come, which no pre/post pair over states can express. The second difference is one of ambition. Dijkstra monads target full functional correctness in a dependently typed host and rely on a solver; **Pledge** targets temporal obligations in plain Haskell, computes a residual rather than a verification condition, and is correspondingly weaker and cheaper.

*Parameterised and graded monads.* ? introduces parameterised monads for Hoare-style state tracking; ? generalises this to graded monads; ? shows effect systems and session types are instances. Pledge carries obligations as value-level annotation fields rather than indexing by effect type at the type level, preserving compatibility with the standard Monad class and **do**-notation.

*Resource management in Haskell.* The bracket combinator lexically scopes cleanup. ResourceT [?], Acquire, and Managed thread finaliser queues through every bind. All three handle the cleanup half of a resource obligation but cannot express protocol obligations, which require explicit future-condition propagation across bind steps, as §5.2’s transaction example shows.

*Linear types.* Linear Haskell [?] enforces “consume exactly once”. Linear types enforce structural obligations (use at most once), whereas future conditions enforce temporal obligations (use in a specified order, or within a specified deadline as in §7). A hybrid combining both could achieve stronger guarantees than either alone.

*Session types.* Session types [??] enforce communication protocols at the type level. The sessiontypes library [?] realises this via an indexed monad. Pledge is more lightweight: it operates within the standard Monad class and is applicable beyond message-passing.

*Typestate.* Typestate [?] attaches a protocol state to a variable’s type and checks transitions statically; ? extend this to aliased objects with access permissions. The obligations are the same

ones **Pledge** targets — an opened file must be closed, a lock released — but the mechanism differs in where the state lives. `Typestate` puts it in the type, so the checker rejects a bad program and the protocol must be fixed before the code is written; **Pledge** puts it in a value, so a bad program compiles and its residual reports what it owes, located traceably once §8 is in place.

*Runtime verification.* Runtime verification [?] checks temporal properties against a concrete execution rather than a model, and is the setting §5 argues **Pledge** realises. ? monitor temporal formulae by rewriting them as events arrive — the residual formula after each event being precisely the obligation that remains, the same move the quotient makes here. MOP [?] generates efficient monitors from a specification attached to program points, and Copilot [?] generates hard-real-time C monitors from a Haskell EDSL, the closest prior system to §7’s bounded intervals. Two things distinguish the present work. First, most monitors are automaton- or rewriting-based and report a Boolean verdict; here the monitor state *is* the specification, in the same algebra the programmer wrote, so a failure names the outstanding obligation. Second, monitor composition is usually external — monitors are attached to a program, and combining them is a matter of running several at once — whereas here composition is the program’s own  $\gg$ , governed by the algebraic laws of §3. The cost, as §1 states plainly, is that **Pledge** observes only what the annotations report and does not yet intervene — closing that gap is §8’s enforcing-mode item.

*Derivatives of regular expressions.* Brzowski derivatives [?] have found application in parsing [??] and model checking; ? refine them into partial derivatives, whose sets stay smaller. Equation (4) extends the construction to complement. What is new here is not the derivative but the use of a *language* quotient — of one specification by another, rather than by a single event — as the propagation rule of a monadic bind, and the two front-ends of §5–§7 that reuse it unchanged.

## 12 Conclusion

We have presented *Pledge*, a Haskell library that realises ?’s proposal of future conditions as a concrete monad transformer: a **Pledge**  $\mathfrak{m}$  *eff* a computation in underlying monad  $\mathfrak{m}$  that carries three annotation fields (*pre*, *post*, *fut*)  $:: \mathit{eff}$ , propagated algebraically through monadic bind via left- and right-quotient operations abstracted as a six-operation **Composable** class.

The design now spans two solid instances built on one theory. RE, with Brzowski derivatives computing both quotients, satisfies fourteen algebraic axioms exactly under two stated side conditions — single-word postconditions, past-closed preconditions — proved in Lean 4 alongside a soundness theorem connecting a discharged residual to the trace a run actually emitted. On top of RE, an LTL front-end reframes **Pledge** as a compositional runtime monitor whose state is the residual itself, and a bounded MTL front-end adds discrete-time deadlines by compiling to the same RE. Beside it, a fourth, disjunctive instance GRE pairs traces with Presburger heap predicates discharged by Z3, giving **Pledge** a genuinely static reading as well as a runtime one, checked over symbolic rather than concrete heaps.

What the design does not yet do is the honest measure of what remains: a violation is reported but not located (§8); the Lean proof burden of a new instance is still manual (§9); and the design has been validated for correctness but not yet measured for cost (§10). A few smaller items sit beside these three: promoting the residual from a reported value to an enforced one by failing bind on  $\emptyset$  (§8); a parallel composition operator, corresponding to the shuffle product on RE, for concurrent programs, which the sequential **Pledge** monad does not yet offer; and hardening the API so that §4.5 and §4.6’s side conditions are unwriteable rather than merely documented, by exposing smart constructors in place of the raw RE constructors in *post* and *pre* slots. None of these require revisiting the theory of §3–§4; each is a self-contained piece of engineering on top of a foundation we have proved correct.